

1 Motivation and Business Case

Theme: cloud-based production-engineering education.

1.1 Problem Statement and Formulation

Students and junior engineers need opportunities to practise responding to production failures before they become responsible for live services. Reading an incident report explains what happened, but does not require the reader to investigate incomplete evidence or decide how to recover a service. In a real incident, a decision can also change the problem itself. An attempted fix may provide temporary relief, create another failure, or make recovery harder.

The learning problem therefore includes both diagnosis and judgement. Learners must decide which signals matter, identify a likely cause, and predict how an intervention will affect the system. They must then assess the result and revise their approach. Google's Wheel of Misfortune supports this practice through team exercises that reenact incidents from previous postmortems [1]. Google's guidance also recommends reviewing drill outcomes to identify lessons and gaps [2].

1.2 Purpose, Target Users, and Proposed Value

OpsReplay is a production-engineering learning platform with three modes: **Learn**, **Challenges**, and **Code Review**. Its primary users are computer science students, along with junior software, DevOps, platform, and site reliability engineers. Learn explains documented failures. Code Review develops the ability to identify risky changes. Challenges provide practice investigating and recovering a simulated service.

OpsReplay adapts this training approach for individual practice in a web browser. Prepared scenarios and automated feedback let learners practise without arranging a team exercise. The simulation models lasting action effects and lets learners replay selected decisions to compare outcomes.

For example, scaling an application with a connection leak may overload a shared database and disrupt another service. A later rollback stops the leak, but queued work still delays recovery. An earlier rollback may avoid that disruption. After an attempt, learners can return to a saved decision point, try another action, and compare recovery time, service impact, and the causal explanation. The intended benefit is stronger reasoning about action order and recovery, which the evaluation in Section 4 will assess.

2 Business Model

All Learn material will remain free, including summaries and links to official sources. A Pro subscription will unlock the team-authored Challenges and Code Review exercises, including their feedback and debriefs. Institutional access will license the same exercise library to a class or training cohort at a charge per seat. The paid value is the authored practice experience.

Initial adoption will focus on students and junior engineers. Content development, cloud hosting, and language-model use are the main costs. Prices and willingness to pay remain unvalidated. The pilot will assess learning value and interest in further exercises, while system measurements will estimate the cost of delivering a session.

3 Architecture and Implementation

3.1 Learning Modes and User Experience

Learn will provide categories, team-written summaries, lessons, and links to official incident reports. Code Review will present a code diff with context. Learners will identify risky lines and explain their concerns, then compare their findings with prepared feedback. They can filter exercises by the programming languages available in the library.

Both Challenges and Code Review will have Easy, Medium, and Hard tiers. Difficulty will reflect evidence ambiguity, service dependencies, and the subtlety of defects. The initial library will stay small. These modes support studying failures, responding to them, and identifying risky changes.

Challenges are single-player exercises and the main development focus. Each begins with an alert and limited evidence. A monitoring dashboard shows metrics, while a resource explorer provides health, logs, events, configuration, and deployments. Learners inspect evidence, form a hypothesis, and apply mitigation until recovery or failure. Metrics and events share one simulated clock, with actions marked on charts so learners can connect decisions to observed changes.

3.2 Evolving Scenario Engine

Each Challenge models only the factors relevant to its failure. Structured definitions specify initial state, permitted actions, time costs, metric rules, event conditions, and recovery or failure criteria. A deterministic state machine uses variables and transition rules so paths can converge or diverge. Lasting state, such as accumulated work, makes action order matter.

For each valid action, the engine applies its effect, advances simulated time, evaluates rules, and records events. Defined time steps capture threshold crossings and delayed effects. New investigation requests can consume time, so an untreated incident can worsen during diagnosis. Metrics derive from state rules and logs use authored templates. Actions and explicit time advances drive the simulation. Rereading evidence, browser waiting, and AI response delays do not advance time. No continuous simulation process is needed.

Table 1 shows a proposed connection-leak Challenge in which checkout and payment share a database. Each path begins from the same state.

Action sequence	Modelled effect
Roll back early	Replace leaking processes and close their connections before shared capacity is exhausted.
Scale, then roll back	Extra instances exhaust connections and disrupt payment. Rollback stops the leak, but queued work must still drain.
Restart application	Leaked connections close and pressure eases. The faulty version remains, so the leak returns.

Table 1: Action order and lasting effects change the recovery path.

The same scenario version and ordered actions, including simulated time advances, produce the same states, metrics, events, and scores.

3.3 Direct Controls and Conversational Interaction

Direct controls let learners choose resources, evidence, and mitigation parameters. An optional large language model (LLM) maps natural-language requests to the same typed operations, such as `get_metric` and `scale_service`. The backend validates parameters and prerequisites, executes the operation, and returns the result for the model to explain. Learners confirm proposed mitigation before execution. Each LLM call receives current visible state, revealed evidence, permitted tools, and relevant history, including manual actions. Unrevealed scenario data stays outside the prompt. The engine owns state, consequences, and scores.

3.4 Proposed AWS Deployment

Figure 1 shows the proposed deployment. Independent sessions allow a class to practise the same Challenge without sharing incident state. Short simulation requests suit on-demand compute, while separate LLM handlers allow limits on slower conversational requests.

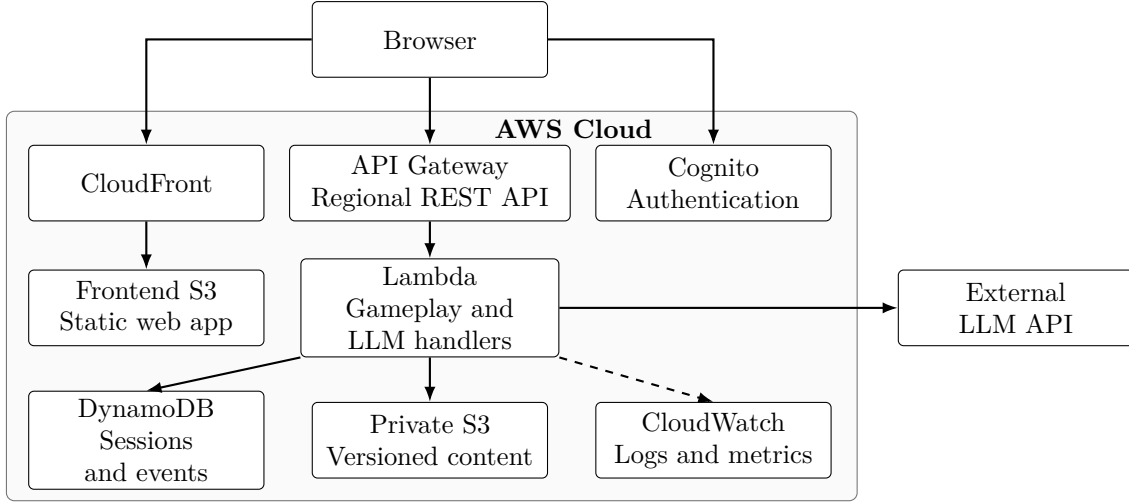


Figure 1: Requests and responses use the solid paths. The dashed arrow shows application monitoring. The API exposes only permitted content and evidence.

S3 and CloudFront will serve a static web application. Cognito will identify users, while the back-end checks session ownership and exercise access. Private S3 will hold versioned content and hidden scenario data. CloudWatch will monitor the application separately from the simulated incident dashboard.

A Regional API Gateway REST API will route requests to stateless Lambda handlers. The message route will stream responses while preserving the full tool-selection, execution, and explanation loop. AWS supports response streaming for REST APIs, while the cheaper HTTP API lacks streaming and has a 30-second integration limit [3]. An early deployment test will verify runtime and regional support. Streaming improves feedback but does not reduce model computation. Lambda remains billed while waiting, even after a client disconnects [4]. Provider timeouts, token limits, and separate concurrency budgets will bound AI use and preserve capacity for direct actions.

DynamoDB fits session-based access without relational joins. It will store progress, review submissions, and conversation history. Each session will have a bounded state record, separate ordered events, and stored request results. A transaction will update state, append an event record, and save the result under a request ID. Retries will retrieve that result without repeating effects. Version checks will reject stale actions, including tools selected before a direct action changed the session. Completed actions remain retrievable if an LLM explanation fails. Replay will create a separate session from a saved checkpoint, preserving the original attempt and scenario version.

This design reduces idle compute and server maintenance, but introduces cold starts, service quotas, and AWS-specific integration. A single-server application with PostgreSQL has fewer deployment components and more flexible queries, but requires provisioned capacity and server maintenance. It may suit sustained traffic better. The proposed serverless design targets intermittent training demand. The engine will remain separate from cloud access code. Local storage adapters will support testing before deployment. An AWS test deployment will then verify authentication, permissions, persistence, and streaming. This separation also supports an equivalent on-premise implementation.

3.5 Implementation Plan

The team will first prove one complete Challenge, then add a small Learn collection and Code Review set. Expansion will follow content validation.

Period (2026)	Planned deliverable
21–28 Sep	Finalise the reference scenario and submit the preliminary report.
29 Sep–11 Oct	Build the engine and test lasting effects, checkpoints, and deterministic replay. Validate AWS transport.
12–25 Oct	Integrate the interface, persistence, replay comparison, debrief, and LLM. Add Learn content and tiered Code Review with language filters.
26 Oct–5 Nov	Validate additional content and run system tests and the learner study.
6–13 Nov	Resolve findings and complete cost analysis, the final report, slides, and demonstration.

Table 2: Proposed delivery schedule, with Challenges receiving most development effort.

3.6 AI Teaching Support

The LLM will offer concept explanations using visible evidence and hints released by the backend. After submission, it can explain reference findings and the debrief. It will not set scores or grade free-text reasoning. AI and hint use will be recorded to distinguish assisted results. Direct controls and prepared feedback remain available if the provider fails.

3.7 Scoring, Debrief, and Replay

Exercises will use original artefacts inspired by documented failures and link to official sources. Challenge scoring will consider investigation effort, recovery time, and cumulative service impact. Reduced impact counts even when the root cause remains. The debrief will explain key and missed evidence, action effects, the root cause, and recommended recovery using recorded events.

After an attempt, learners can replay from a few authored checkpoints and compare paths from the same state. The comparison will show recovery time, service impact, and why outcomes differ. The original history and score will remain unchanged. Replay is informed practice after the debrief, so its results will be labelled separately from the first attempt.

4 Evaluation Plan

Results will identify scenario versions, deployment settings, and sample size.

4.1 System Correctness, Reliability, and Performance

- **Correctness.** Repeat fixed actions and compare states, metrics, events, and scores. Test action order, delayed effects, temporary relief, and checkpoint restoration. Verify that replay preserves the first attempt, interface values match state, and content references and answers are valid.
- **Access and reliability.** Test ownership, paid access, hidden evidence, stale actions, duplicate requests, interrupted writes, and provider failures. Retries must not repeat effects or score penalties.
- **Performance.** Measure median and 95th-percentile latency, throughput, errors, and database throttling at 10, 50, and 100 independent sessions. Record action mix and think time. These are proposed test loads.

4.2 LLM Accuracy, Latency, and Cost

A labelled request set will specify context and acceptable tools and arguments. Tests will measure selection accuracy, invalid calls, unsupported claims, answer leakage, and teaching accuracy. For each model, we will record median and 95th-percentile latency for the full conversational loop and token cost per session. Engine load tests will exclude provider delay, with separate real-provider tests under a fixed budget. Simulated responses will be labelled.

4.3 User Evaluation

We propose a formative pilot with 5–10 computer science students or junior engineers. Each will investigate an incident, read its debrief, and replay a selected decision. For another incident of similar difficulty, the learner will read a static explanation that includes alternative actions and outcomes. We will balance assignment and order as far as the sample permits, allow equal study time, and provide equivalent causal information. Direct controls without AI will isolate the simulation and replay workflow from teaching assistance.

Questions before and after each condition will assess diagnosis and causal reasoning using a common rubric. Before replay, learners will predict how a different action changes recovery. Afterward, they will explain the result and apply the lesson to a new decision. We will record completion, harmful actions, and explanation quality. Brief tasks will check Learn, Code Review, and optional AI support. Participants will complete the System Usability Scale (SUS) [5] and a short interview on usability and replay usefulness.

Results will report individual scores, paired differences, and usability problems. The pilot can guide improvement and provide early learning evidence. It cannot establish broad effectiveness, long-term retention, or the separate effect of replay within the full workflow.

4.4 Cloud Cost and On-Premise Comparison

The preliminary choice of serverless hosting assumes intermittent demand. We will compare steady use and class-sized bursts with an on-premise deployment of the same service. The final report will use measured workloads and published rates. AWS costs will include API requests, compute, database, storage, transfer, authentication, and monitoring. On-premise costs will include suitable hardware spread over its useful life, power, networking, maintenance, and staff time. Both will use the same external LLM and include its cost. We will report cost per completed session, capacity, and availability assumptions. Credits will be separate, without assuming cloud hosting is cheaper.

5 Conclusion

OpsReplay combines free learning with authored practice. Its main focus is an evolving incident with lasting consequences and replay of recovery decisions. The team will prove one complete Challenge before expanding the library and evaluating usability, learning, reliability, and cost. The plan assumes access to testers and an LLM provider with function calling. Exercise counts, available languages, scoring weights, model choice, and prices remain open.

References

- [1] Google, “Postmortem Culture: Learning from Failure,” *The Site Reliability Workbook*.
<https://sre.google/workbook/postmortem-culture/>.
- [2] Google, “Incident Response,” *The Site Reliability Workbook*.
<https://sre.google/workbook/incident-response/>.
- [3] AWS, “Choose Between REST APIs and HTTP APIs” and “Quotas for HTTP APIs,” *API Gateway Developer Guide*.
- [4] AWS, “Stream the Integration Response” and “Response Streaming for Lambda Functions,” *AWS Documentation*.
- [5] J. Brooke, “SUS: A ‘quick and dirty’ usability scale,” in *Usability Evaluation in Industry*, 1996, pp. 189–194.

Web references accessed 21 September 2026.

AI-use declaration. The team developed the project concept and selected the design direction. AI assisted with design evaluation, reference checks, report drafting, language refinement, and LaTeX preparation. This work clarified the distinction between repeatable simulation results and learning outcomes that require user evaluation.